Spring 2026

ME/CS/ECE 759

Final Project Report

University of Wisconsin–Madison

FlashAttention in Raw CUDA

Gaurav Batra
gbatra3@wisc.edu

May 5, 2026

Abstract

When a transformer attends over long sequences, the bottleneck is rarely arithmetic: it is the cost of writing a full $N \times N$ score matrix to GPU global memory and reading it back for softmax. I implemented a FlashAttention-style forward pass from scratch in CUDA C++, replacing that round trip with tiled shared-memory kernels and an online softmax recurrence [1, 3]. I benchmarked five kernel families on an NVIDIA Tesla T4 on **Google Cloud Platform**: a three-kernel naive baseline, tiled FP32 FlashAttention (v1), causal FlashAttention, a warp-centric FP32 experiment (v2), and an FP16 WMMA score path. Tiled Flash cuts latency by roughly 1.9× at N=8192 and drops global reads by about one order of magnitude at N=4096. Causal masking gives the best wall-clock time in my runs ($\approx$ 30.6 ms at N=8192). The WMMA path reaches $\approx$ 50 ms non-causal with the smallest HBM footprint of any variant. The v2 prototype matches the reference but runs slower than even the naive baseline at large N. *All timings and NCU data in this report come from a Tesla T4 on* ***Google Cloud Platform****. For measured host and GPU specifications (instance type, bandwidth, driver, CUDA, NCU), see here*

Final Project git repo: `https://github.com/batra98/me759-flashattention`

(Interactive write-up: `https://batra98.github.io/me759-flashattention`.)

Video Walkthrough: `https://youtu.be/BzBU7hClPFo`.

Contents

1 General information **2**

2 Problem statement **2**

3 Solution description **2**

3.1 Hardware and how I measured . . . 2
3.2 Repository layout and build . . . 3
3.3 Shared utilities (`attention_utils.cuh`) . . . 3
3.4 CLI driver (`main.cu`) . . . 3
3.5 Kernel modules (overview) . . . 4
3.6 Naive baseline (three kernels) . . . 4
3.7 FlashAttention v1 (tiled FP32) . . . 5
3.8 Causal FlashAttention (FP32) . . . 5
3.9 Experimental v2 kernel (`flash_v2`) . . . 6
3.10 WMMA scores (Tensor Cores) . . . 6
3.11 Companion website . . . 7

4 Overview of results. Demonstration of your project **7**

4.1 Latency and speedup . . . 7
4.2 Memory traffic (Nsight Compute) . . . 8

5 Deliverables. Building & running your project **10**

5.1 Clone and compile . . . 10
5.2 Benchmarks and NCU profiling . . . 10

6 Conclusions and Future Work **10**

1 General information

- **Name:** Gaurav Batra
- **Email:** gbatra3@wisc.edu
- **Home department:** Computer Sciences
- **Status:** MSCS student
- **Teammate (if applicable):** Not applicable (solo project).
- **Code release:** I release the ME/CS/ECE 759 Final Project code as open source under a BSD 3-Clause license for unfettered use by any interested party.

2 Problem statement

Attention is the engine of modern language models, but the standard implementation has a quiet performance problem. For every forward pass, it writes the full $N \times N$ score matrix $S = QK^\top/\sqrt{d}$ to high-bandwidth memory (HBM), then immediately reads it back to compute softmax [2]. At sequence lengths like $N = 4096$ or $N = 8192$, that traffic completely dominates the wall-clock time; the GPU is mostly waiting on memory, not computing.

Dao et al. [1] showed that you can avoid materializing S at all by tiling Q, K, and V into blocks that fit in fast on-chip shared memory (SRAM), and tracking softmax statistics incrementally as you go. The result is the same attention output, but with $O(N)$ global memory traffic instead of $O(N^2)$.

I reimplemented that idea from scratch in raw CUDA, without relying on any closed-source kernel libraries. My goals were: (1) check that my kernels match a reference implementation, (2) measure how much latency and HBM traffic change on a real T4, and (3) try a few design variants (causal masking, a warp-centric FP32 layout, and Tensor Core scores) so I could see the tradeoffs between IO, occupancy, and arithmetic directly in the data.

3 Solution description

3.1 Hardware and how I measured

I ran all experiments on an **NVIDIA Tesla T4** (sm_75, Turing) on **Google Cloud Platform**. For the measured host and GPU table (instance type, memory bandwidth, L2, shared memory limits, driver, CUDA toolkit, Nsight Compute version), see README.md in the repository.

I did not use the department **Euler** cluster for the final numbers: the instruction-partition software stack there paired **CUDA 12.2** with a **GCC 15** host compiler that `nvcc` rejects without non-default overrides and **Nsight Compute** (`ncu`) for the HBM sweep is invoked with `sudo` in `run_ncu_profile.sh`, which Euler does not grant for routine jobs.

3.2 Repository layout and build

I use **CMake** to build a single executable `flash_attn` for sm_75 (`CMAKE_CUDA_ARCHITECTURES=75`). I compile every `.cu` file with `-O3`, `--extended-lambda`, `--expt-relaxed-constexpr`, `-lineinfo`, and `--ptxas-options=-v`, then link one binary so the host driver and kernels stay matched.

```
me759-flashattention/
  final_report.tex                % this PDF (source at repo root)
  FinalProject/
    README.md                     % build, bench, NCU, plots (grader entry point)
    CMakeLists.txt
    src/
      attention_utils.cuh         % shared macros, timer, constants
      main.cu                     % CLI, allocations, correctness vs benchmark
      naive_attention.cu        % three-kernel naive + causal QKT
      flash_attention_v1.cu       % tiled FP32 Flash (templates BR, BC, D)
      flash_attention_v1_causal.cu
      flash_attention_v2.cu
      flash_attention_v3_wmma.cu
    build/                        % out-of-tree CMake output (gitignored)
    data/results/                 % timing.csv, hbm_traffic.csv, plots/*.png
    benchmarks/
      run_bench.sh
      run_ncu_profile.sh
    python/
      plot_results.py             % reads data/results CSVs, writes PNGs
    docs/                         % GitHub Pages site + docs/assets figures
```

3.3 Shared utilities (`attention_utils.cuh`)

I kept boilerplate in one header instead of duplicating it across every `.cu` file:

- `CUDA_CHECK` wraps every runtime call and prints a clear error message with file and line number if something goes wrong.
- `GpuTimer` wraps two `cudaEvent_t` handles and returns elapsed milliseconds between `Start` and `Stop`.
- **Host helpers:** `fill_random` fills Q, K, V from a fixed seed so every kernel sees the same inputs; `rmse` computes the root-mean-square error between two output arrays.
- **Tile constants:** `D_HEAD=64`; Flash v1 uses `BR=BC=32`; v2 uses `BR_V2=16`, `BC_V2=32`; WMMA uses `BR_WMMA=BC_WMMA=16`. All matrices are row-major with shape $[N, d]$.

3.4 CLI driver (`main.cu`)

`main.cu` parses `--mode`, `--target`, `--seq_len`, `--d_head`, `--warmup`, `--iters`, and `--csv`. It allocates and uploads Q, K, V to the device, then branches into one of two paths:

- **Correctness mode** (`--mode correctness --target X`) allocates an extra $N \times N$ buffer `dS` for the naive score matrix. For non-causal targets, the naive reference runs first into `dO_ref`, then the kernel under test runs into `dO_tst`. RMSE must be below 10^{-3} for FP32 kernels and $5 \cdot 10^{-3}$ for WMMA.
- **Benchmark mode** warms up the kernel, then averages `iters` timed runs with `GpuTimer`. The result is printed as plain text or as a CSV line (`mode,N,d,ms`) with `--csv`.

I route kernels through `run_target`: `flash` calls `flash_attention_v1`, `flash_causal` calls `flash_attention_v1_causal`, and the WMMA modes call their wrappers. Naive modes bypass that helper because they still allocate the full $N \times N$ `dS` buffer.

```
static void run_target(const char* mode,
    const float* dQ, const float* dK, const float* dV,
    float* dO, int N, int d)
{
    if      (!strcmp(mode, "flash"))
        flash_attention_v1(dQ, dK, dV, dO, N);
    else if (!strcmp(mode, "flash_causal"))
        flash_attention_v1_causal(dQ, dK, dV, dO, N);
    else if (!strcmp(mode, "flash_v2"))
        flash_attention_v2(dQ, dK, dV, dO, N);
    else if (!strcmp(mode, "flash_wmma"))
        flash_attention_v3_wmma(dQ, dK, dV, dO, N);
    // ... flash_wmma_db, error handling
}
```

3.5 Kernel modules (overview)

Table 1 lists each source file. For each kernel I wrote a small host wrapper that sets the launch grid and instantiates the device template with fixed shapes (for example $(B_r, B_c, d) = (32, 32, 64)$ for v1).

Table 1: Source files, host APIs, and primary device kernels.

File	Role
`naive_attention.cu`	Host `naive_attention`, `naive_attention_causal`. Kernels: `qkt_kernel`, `qkt_kernel_causal`, `softmax_kernel`, `pv_kernel`.
`flash_attention_v1.cu`	Host `flash_attention_v1`. Device `flash_attn_fwd` $\langle B_r, B_c, d \rangle$. Shared $Q/K/V$ tiles; online softmax (Listing 1).
`flash_attention_v1_causal.cu`	Host `flash_attention_v1_causal`. `flash_attn_fwd_causal` $\langle B_r, B_c, d \rangle$. Skips future-only tiles; boundary masks (Listing 2).
`flash_attention_v2.cu`	Host `flash_attention_v2`. Template `flash_attn_fwd_v2`; `__launch_bounds__(512,2)` (Listing 3).
`flash_attention_v3_wmma.cu`	Host `flash_attention_v3_wmma`, `flash_attention_v3_wmma_db` (sm_75). Template `flash_attn_fwd_wmma`; WMMA 16×16 tiles (Listing 4).

3.6 Naive baseline (three kernels)

The naive path follows the textbook definition directly: compute $S = QK^\top/\sqrt{d}$ and write the entire matrix to HBM, run a row-wise softmax pass over S, then compute

$O = PV$. The causal variant masks the upper triangle (setting those logits to -10^{30}) before softmax so the output matches the lower-triangular attention pattern. That gives me a correct reference and the worst-case HBM traffic story.
The QKT kernel is a straightforward dot product per (row, col) pair; the causal variant short-circuits upper-triangular entries:

```
__global__ void qkt_kernel(..., float* S, int N, int d) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row >= N || col >= N) return;
    float acc = 0.f;
    for (int k = 0; k < d; ++k)
        acc += Q[row * d + k] * K[col * d + k];
    S[row * N + col] = acc * rsqrtf((float)d);
}

__global__ void qkt_kernel_causal(..., float* S, int N, int d) {
    // ...
    if (col > row) { S[row * N + col] = -1e30f; return; }
    // ... same dot product as dense qkt_kernel
}
```

3.7 FlashAttention v1 (tiled FP32)

FlashAttention avoids ever storing S in HBM. I use tile sizes $B_r = B_c = 32$ with $d = 64$, so the working Q, K, and V tiles fit in ≈ 24 KB of shared memory. Instead of writing a row of S and reading it back for softmax, I track the running row maximum m_i and normalizer ℓ_i in registers and rescale the partial output whenever a new tile raises the maximum. Milakov and Gimelshein [3] proved this matches a two-pass softmax numerically. Listing 1 shows the rescale I use after each inner tile (from `flash_attention_v1.cu`).

Listing 1: Online softmax rescale and partial PV blend (FlashAttention v1).

```
float m_new = fmaxf(m_i, m_tilde);
float alpha = expf(m_i - m_new);
float beta  = expf(m_tilde - m_new);
for (int k = 0; k < d; ++k) {
    float pv_k = 0.0f;
    for (int jj = 0; jj < bc_actual; ++jj)
        pv_k += S_row[jj] * V_smem[jj][k];
    O_acc[k] = alpha * O_acc[k] + beta * pv_k;
}
m_i = m_new;
l_i = alpha * l_i + beta * l_tilde;
```

Each iteration rescales the running output by `alpha` and blends in the new tile's contribution weighted by `beta`. When the outer loop finishes, the accumulator holds the softmax-weighted output without ever materializing S in global memory.

3.8 Causal FlashAttention (FP32)

For autoregressive models, position i should only attend to $j \leq i$. In `flash_attention_v1_causal.cu` I skip entire K/V column tiles when they lie entirely in the future for the current Q tile. On tiles that cross the diagonal, I mask logits with $j > i$ before the

softmax update. For large N that is roughly half the work of the non-causal kernel, which shows up in both latency and NCU reads.
The early exit in the outer loop over K/V tiles is the main savings: once the start column of a tile is past the last query row in the current Q block, no further tiles can contribute.

Listing 2: Skipping future-only K/V tiles (causal Flash, excerpt).

```
for (int j = 0; j < Tc; ++j) {
    const int kv_start = j * BC;
    // ...
    if (kv_start > q_row_max_in_block)
        break;   // entire tile is in the future for this Q block
    // load K_smem, V_smem, compute logits, online softmax update ...
}
```

3.9 Experimental v2 kernel (`flash_v2`)

I wrote `flash_attention_v2.cu` to try a different thread layout: $B_r = 16$, $B_c = 32$, 512 threads per block, fused score and softmax buffers in shared memory, and `__launch_bounds__(512,2)` after an earlier 1024-thread prototype used ≈ 41 KB of shared memory. The hope was better occupancy on T4; as the results show, it did not pay off.
Listing 3 shows the launch hint, warp shuffle helper, and shared buffers for scores plus running softmax state (one warp per query row within the B_r rows of the Q tile).

Listing 3: `flash_attn_fwd_v2`: occupancy hint, warp reduce, shared tiles.

```
static __device__ inline float warp_reduce_sum(float v) {
    for (int offset = 16; offset > 0; offset >>= 1)
        v += __shfl_down_sync(0xffffffffu, v, offset);
    return v;
}

template <int BR, int BC, int D>
__global__ void __launch_bounds__(512, 2)
flash_attn_fwd_v2(const float* Q, const float* K, const float* V,
                  float* O, int N)
{
    __shared__ float Q_smem[BR][D];
    __shared__ float K_smem[BC][D], V_smem[BC][D];
    __shared__ float tile_scr[BR][BC];  // logits, then softmax weights
    // ... online softmax + PV accumulation ...
}
```

3.10 WMMA scores (Tensor Cores)

`flash_attention_v3_wmma.cu` calls WMMA (`m16n16k16`) for 16×16 score tiles with FP16 operands on the T4's Tensor Cores, while I keep softmax state and the output in FP32 [4].
The `flash_wmma_db` entry point is the same kernel on sm_75; a real async double-buffer path would need sm_80 or newer.
Listing 4 is the inner part of the score computation: I convert Q and K tiles to half precision in shared memory, fill WMMA fragments, and accumulate the 16×16 tile of logits into `Stile` before softmax.

Listing 4: WMMA $QK^\top$ tile (FP16 matmul, FP32 accumulator).

```
wmma::fragment<wmma::matrix_a, 16, 16, 16, half, wmma::row_major> a_frag;
wmma::fragment<wmma::matrix_b, 16, 16, 16, half, wmma::col_major> b_frag;
wmma::fragment<wmma::accumulator, 16, 16, 16, float> c_frag;
wmma::fill_fragment(c_frag, 0.0f);
for (int kk = 0; kk < D; kk += 16) {
    wmma::load_matrix_sync(a_frag, &Qh[0][kk], D);
    // pack B from Kh into Bpack, then:
    wmma::load_matrix_sync(b_frag, Bpack, 16);
    wmma::mma_sync(c_frag, a_frag, b_frag, c_frag);
}
wmma::store_matrix_sync(&Stile[0][0], c_frag, BC, wmma::mem_row_major);
```

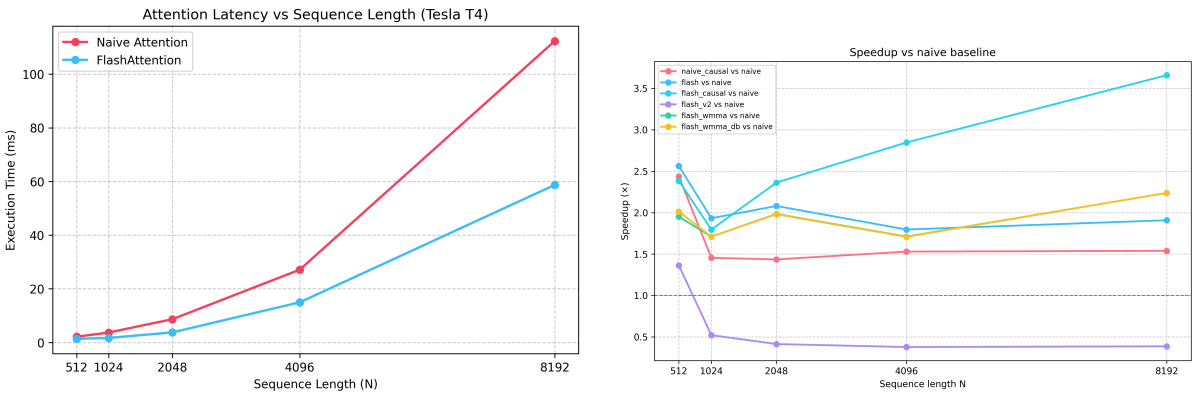
3.11 Companion website

I also published a visual walk-through of the memory hierarchy at https://batra98.github.io/me759-flashattention, mainly for readers who want intuition beyond the CUDA in this report.

4 Overview of results. Demonstration of your project

I benchmarked single-head attention with $d = 64$ and $N \in \{512, 1024, 2048, 4096, 8192\}$. Timings come from `FinalProject/data/results/timing.csv`; memory traffic from `hbm_traffic.csv` in the same folder. In each subsection I put the figures and tables first, then explain how I read them.

4.1 Latency and speedup



(a) End-to-end latency vs. N.

(b) Speedup relative to `naive`.

Figure 1: Results from `FinalProject/data/results/timing.csv`.

Table 2: Mean latency (ms), d=64, Tesla T4.

Kernel	N=4096	N=8192
`naive`	27.12	113.55
`naive_causal`	17.92	73.66
`flash`	15.05	58.68
`flash_causal`	9.52	30.65
`flash_v2`	66.33	252.88
`flash_wmma`	15.79	50.19
`flash_wmma_db`	15.79	50.18

What stands out in Figure 1a is the gap that opens between `naive` and the tiled kernels as N grows. At $N = 512$ everything is within a factor of two; by $N = 8192$ the tiled paths have pulled away. That is what I expected from the IO story: naive pays for the full $N \times N$ score matrix in HBM, while tiled Flash does not.

In Table 2, `flash` is 1.93× faster than naive at $N = 8192$ (58.7 ms vs 113.6 ms), and `flash_causal` wins overall at 30.6 ms. Causal is faster not only because it does less math, but because I skip future K/V tiles, so I load roughly half the data of the non-causal kernel.

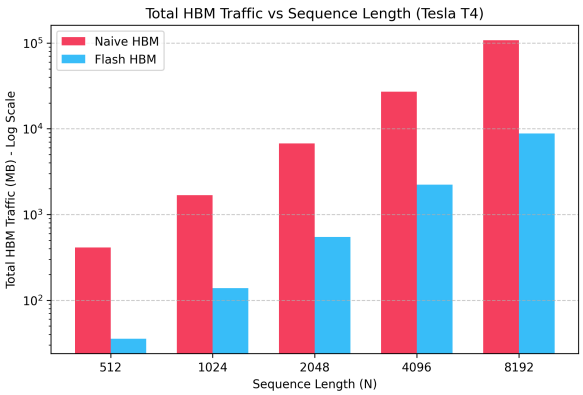
`flash_v2` is the outlier in the wrong direction: 252.9 ms at $N = 8192$, slower than even naive, even though I verified correctness. The memory traffic section below shows why.

WMMA narrows the gap to `flash_causal` somewhat (50.2 ms at $N = 8192$). It beats non-causal Flash but not causal Flash, because I never fused the tile-skipping logic into the WMMA kernel. `flash_wmma_db` matches WMMA on T4; without an Ampere-style async copy pipeline, the double-buffer entry point does nothing extra here.

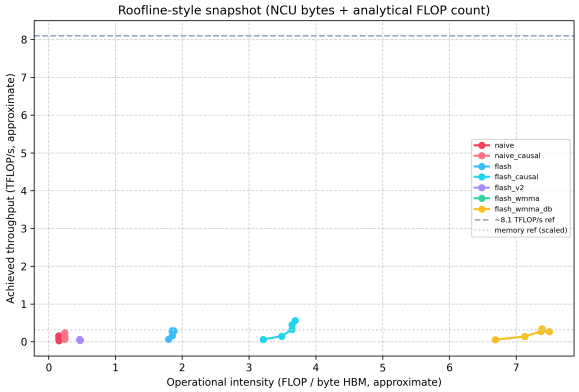
4.2 Memory traffic (Nsight Compute)

Table 3: NCU global loads and stores (MB), N=4096.

Kernel	Read	Write
`naive`	25,835.5	1,163.8
`naive_causal`	15,943.7	1,164.0
`flash`	2,211.8	8.39
`flash_causal`	1,116.2	8.39
`flash_v2`	4,403.2	8.39
`flash_wmma`	537.9	8.39
`flash_wmma_db`	537.9	8.39



(a) Total HBM traffic (read + write; log y).



(b) Roofline-style snapshot (approx. TFLOP/s vs operational intensity).

Figure 2: NCU traffic (`hbm_traffic.csv`) and roofline diagnostic (`FinalProject/python/plot_results.py`).

Table 4: NCU global loads and stores (MB), N=8192.

Kernel	Read	Write	Total
`naive`	103,352.3	4,663.5	108,015.8
`naive_causal`	63,785.0	4,663.7	68,448.7
`flash`	8,816.6	16.78	8,833.4
`flash_causal`	4,423.7	16.78	4,440.5
`flash_v2`	17,582.1	16.78	17,598.9
`flash_wmma`	2,201.6	16.78	2,218.4
`flash_wmma_db`	2,201.6	16.78	2,218.4

The traffic tables 3 and 4 match what I saw on the timer. At $N = 4096$, naive reads ≈ 25.8 GB and writes ≈ 1.14 GB; `flash` reads ≈ 2.21 GB and writes only 8.4 MB. That is about 12× fewer reads and orders of magnitude fewer writes, because I stopped shuttling the full S matrix through HBM. Writes tell the same story: naive must dump the $N \times N$ scores; every tiled variant only writes O, so they all land near the same ≈ 8.4 MB.

`flash_causal` reads about half as much as `flash` at $N = 8192$ (4.4 GB vs 8.8 GB), which directly reflects the tile-skipping: for a given Q tile, all K/V tiles to its right are simply never loaded.

`flash_v2` is the failure I learned the most from. It reads ≈ 17.6 GB at $N = 8192$, roughly twice `flash`, even though I still fused softmax in SRAM. I chose $B_r = 16$ instead of 32, so the outer loop over Q runs twice as many times; each pass still streams all N/B_c tiles of K/V. That doubles K/V traffic versus v1 and explains why the kernel falls behind naive despite correct outputs. It drove home that IO efficiency means matching tile shapes to reuse, not only "fusing" softmax.

WMMA reaches the lowest read volume at $N = 8192$ (≈ 2.2 GB), below even `flash_causal`. FP16 operands are smaller than FP32, so the same 16×16 score tile loads fewer bytes through L1TEX. Combined with Tensor Core throughput for the $QK^\top$ step, WMMA improves on `flash` in both traffic and runtime.

5 Deliverables. Building & running your project

Section 3.2 and Table 1 describe the layout of the repo I submitted. The commands below are what I used (and what you can use) to rebuild the binary, rerun timings and NCU, and regenerate the plots from the CSVs.

5.1 Clone and compile

```
git clone https://github.com/batra98/me759-flashattention.git
cd me759-flashattention/FinalProject
mkdir -p build && cd build
cmake .. && make -j"$(nproc)"
```

5.2 Benchmarks and NCU profiling

```
./flash_attn --mode correctness --target flash --seq_len 2048
bash ../benchmarks/run_bench.sh ./flash_attn ../data/results
sudo bash ../benchmarks/run_ncu_profile.sh ./flash_attn \
    ../data/results/hbm_traffic.csv
```

6 Conclusions and Future Work

To rebuild the binary, rerun benchmarks and NCU, and regenerate figures from the CSVs, follow `FinalProject/README.md` (same steps as Section 5, with paths relative to `FinalProject/`).

The clearest lesson for me is that tiling alone does not win: you have to match tile shapes to how often you reload K and V. The naive baseline shows $O(N^2)$ HBM traffic crushing runtime at large N. My FlashAttention v1 kernel fixes that by never writing the full score matrix to HBM, and the $\approx 1.9\times$ speedup at $N = 8192$ lines up with the traffic drop in NCU. Causal masking goes further by skipping work the dense problem never needed, which gave the best latency in my sweep. WMMA cuts reads even further and speeds up the non-causal path, but I did not fuse causal skipping into it yet, so it still loses to causal FP32 on wall-clock time.

In `flash_v2` NCU shows twice the reads of v1 because halving B_r doubles how often I sweep K/V. That is easy to overlook when you are tuning occupancy. It reminded me that the bottleneck is often memory access patterns, not whether softmax is "fused" in principle.

Next, I would fuse causal masking into the WMMA path, revisit `flash_v2` with v1-sized tiles for Q, and maybe explore KV-cache-aware inference. Longer term, multi-GPU attention and FlashAttention-2/3-style scheduling [1] are on my list.

References

[1] Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.* NeurIPS 2022. https://arxiv.org/abs/2205.14135

[2] Vaswani, A. et al. (2017). *Attention Is All You Need.* NeurIPS 2017. https://arxiv.org/abs/1706.03762

[3] Milakov, M. & Gimelshein, N. (2018). *Online normalizer calculation for softmax.* arXiv preprint. https://arxiv.org/abs/1805.02867

[4] NVIDIA. *CUDA C++ Programming Guide: Warp Matrix Functions (WMMA).* https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#warp-matrix-functions